第 3 章　无线自组网路由协议仿真

本章重点阐述无线自组网路由基本原理，包含路由协议设计思路和性能评价、真实地图生成仿真场景、AODV 和 DSR 的工作过程及性能分析等，旨在让读者了解无线自组网路由协议的设计原理和应用，并能用 NS-2 进行评价分析。

3.1 预备知识

3.1.1 无线自组织网络概述

无线自组织网络(Ad Hoc Network)是一种特殊类型的无线网络，它不依赖于任何固定的基础设施，如基站或接入点。在这种网络中，移动节点动态地创建并管理网络连接，使得网络可在没有预先存在的网络基础设施的情况下快速部署。以下是无线自组织网络的一些关键特征：

①**自组织性**：网络中的节点能够自动配置和重新配置，以维持网络的最佳性能；

②**动态性**：节点的移动性导致网络拓扑频繁变化；

③**无中心**：网络中没有固定的中心节点，所有节点都是平等的，并且可能同时充当路由器和终端；

④**多跳路由**：由于无线通信范围有限，节点通常需要通过其他节点中继来传输数据；

⑤**有限的资源**：节点通常受到处理能力、存储空间和电源的限制。

无线自组织网络主要应用在以下场景：

①**军事通信**：在战场环境中，士兵和设备需要快速建立通信网络；

②**紧急响应**：在自然灾害或其他紧急情况下，固定通信设施可能受损，无线自组织网络可以迅速建立。

③**移动会议**：参与者可以通过自组织网络进行无线通信。

④**传感器网络**：在环境监测、智能交通系统等领域，传感器节点可以自组织成网络进行数据收集和传输。

无线自组织网络的研究和应用对于开发新型无线通信技术和协议具有重要意义，特别是在需要快速部署和高度灵活性的场合。在无线自组织网络中，需要重点解决路由问题、能量管理、网络管理、拥塞控制、安全性等问题。本章主要仿真分析无线自组网路由协议，接下来将介绍后续实验使用的路由协议。

3.1.2 无线自组网路由协议分类

无线自组织网络路由协议大致可分为以下几类，每类中都有一些典型的协议：

①**先验式路由协议**(Proactive Routing Protocols)：优化链路状态路由(Optimized Link State Routing Protocol, OLSR)，节点通过洪泛链路状态信息来维护整个网络的拓扑图，并使用迪杰斯特拉算法计算最短路径；目的地序列距离矢量路由(Destination-Sequenced Distance-Vector Routing, DSDV)，周期性地广播路由更新来维护路由表，在每个路由更新中包含一个序列号，以确保路由信息的最新性并避免路由环路；分区多播路由(Partitioning Multicast Routing, PMR)，用于在自组网中支持多播通信；

②**按需路由协议**(Reactive Routing Protocols)：动态源路由(Dynamic Source Routing, DSR)，源节点在数据包中包含完整的路由信息，不需要周期性地交换路由信息；目的序列距离矢量路由(Ad Hoc On-Demand Distance Vector Routing, AODV)，源节点在需要时发送路由请求(Route Request, RREQ)，目的节点或中间节点回应路由回复(Route Reply, RREP)；

③**混合路由协议**(Hybrid Routing Protocols)：区域路由协议(Zone Routing Protocol, ZRP)，结合了先应式和按需路由协议的特点，节点维护一个局部区域内的路由信息，而跨区域通信则按需建立路由；簇头网关交换路由协议(Cluster-Head Gateway Switch Routing, CGSR)，将网络划分为簇，每个簇有一个簇头，簇头之间进行路由；

④**地理位置路由协议**(Geographic Routing Protocols)：贪婪周边无状态路由(Greedy Perimeter Stateless Routing, GPSR)，利用节点的地理位置信息，贪婪地选择下一跳节点，直到无法找到更接近目的地的节点；受限边界转发(Constrained Border Forwarding, CBF)，在 GPSR 的基础上增加了对障碍物的处理。

上述不同类型的路由协议具有各自的特点，适用于不同的应用场景和需求，如先验式路由协议适合节点移动性较低的网络，而按需路由协议适合节点移动性较高的网络。混合路由协议则试图结合两者的优点。地理位置路由协议则适用于节点能够获取自身地理位置信息的网络环境。在选择路由协议时，需要根据网络的具体需求和条件来决定。

本章后续实验将针对 AODV 和 DSR 展开分析，其他更多路由协议的分析读者可查阅资料自行完成。

3.1.3 AODV 路由工作过程

AODV 允许节点仅在需要时才建立和维护路由，从而节省了网络资源。以下是 AODV 路由协议的详细工作过程：

①**路由发现**：当源节点 S 需要向目的节点 D 发送数据，但它没有到 D 的路由信息时，它会启动路由发现过程。源节点 S 广播一个 RREQ 消息。RREQ 包含源节点地址、目的节点地址、序列号、跳数计数器等信息。每个接收到 RREQ 的中间节点检查是否已经有一条到目的节点 D 的路由。如果没有，它会记录下从源节点 S 到自己的反向路径，并继续广播 RREQ。

②**RREQ 传播**：RREQ 消息在网络中传播，每个节点都会增加跳数计数器的值，并记录下到达自己的前一跳节点。如果一个节点之前已经收到过具有相同序列号的 RREQ，它会丢弃这个重复的 RREQ，以避免循环。当 RREQ 到达目的节点 D 或一个拥有到 D 的有效路由的中间节点时，路由发现过程结束。

③**路由回复**：目的节点 D 或拥有有效路由的中间节点(称为响应节点)会单播发送一个 RREP 消息回源节点 S。RREP 消息包含目的节点地址、源节点地址、序列号、跳数、生命周期等信息。RREP 沿着 RREQ 建立的反向路径传播回源节点 S。在 RREP 返回源节点 S 的过程中，沿途的每个节点都会在路由表中建立到目的节点 D 的路由条目，并记录下到目的节点的下一跳。

④**数据传输**：一旦源节点 S 接收到 RREP，它就可以开始向目的节点 D 发送数据包。数据包会根据路由表中的信息被逐跳转发到目的节点 D。

⑤**路由维护**：如果路由中的某个链路失效，节点会发送 RERR（Route Error）消息通知上游节点。收到 RERR 消息的节点会更新自己的路由表，移除失效的路由，并可能启动新的路由发现过程。路由条目有一个生命周期，如果在这个时间内没有数据传输，路由条目会被删除，以节省资源。

3.1.4 DSR 路由工作过程

类似 AODV，DSR 允许节点在无需维护全局路由信息的情况下发送数据包。DSR 主要包含路由发现和路由维护两个过程，具体如下：

(1)路由发现

当源节点需发送数据包到目的节点且无有效路由信息时，DSR 会启动路由发现过程，具体为：①源节点广播 1 个 RREQ，其包含源节点地址、目的节点地址和唯一标识符；②中间节点接收到 RREQ 后，检查是否已经处理过该 RREQ(避免环路)。若没有，节点将记录下它从哪个邻居节点接收到这个 RREQ，并将该邻居节点地址添加到 RREQ 中的路由记录列表中，然后继续广播 RREQ；④当目的节点收到 RREQ 或中间节点缓存中有到目的节点的路由时，节点单播 1 个 RREP 回源节点。RREP 包含从目的节点到源节点的路由信息，该信息就是 RREQ

中的路由记录列表的逆序；⑤源节点收到 RREP 后，会将这条新发现的路由存储在路由缓存中，并开始使用这条路由发送数据包。

(2)路由维护

路由维护过程包括：①**失效检测**。当某节点试图通过某条链路发送数据包，但失败了(如没收到确认)，它会认为这条链路失效；②**RERR 消息传播**。发现链路失效的节点生成一个 RERR 消息，并将其发送给那些可能使用这条失效链路的上游节点；③**路由修复**。收到 RERR 消息的节点会尝试从其路由缓存中找到替代路由，若找到则会更新数据包头部信息，并继续发送该数据包。若未找到替代路由则将 RERR 消息继续向上游传播；④**路由重发现**。若无可用的缓存路由，源节点可能需重新发起路由发现过程，以找到新的到目的节点的路由。

DSR 协议适合于那些节点移动性强、带宽有限、网络拓扑变化频繁的无线自组网（Ad hoc）环境。由于它不要求节点维护全局路由信息，因此它能够有效地减少控制开销。

3.1.5 SUMO 生成 NS-2 仿真场景

(1)安装 SUMO 工具及其依赖库

①**安装 SUMO 工具**。在 Ubuntu 系统中同时按下快捷键“**Ctrl+Alt+T**”打开终端，然后在终端中逐条执行下述命令：

```
$sudo add-apt-repository ppa:sumo/stable
$sudo apt-get update
$sudo apt-get install sumo sumo-tools sumo-doc
```

②**安装 SUMO 库**。从链接 https://sourceforge.net/projects/sumo/files/sumo/version%200.32.0/ 处下载“**sumo-src-0.32.0.tar.gz**”压缩包，将其拷贝至“**/home/wiotlab/WIoTLab/tools**”目录中并解压(假定文件夹名为“**sumo**”)，然后在“**~/.bashrc**”文件中添加环境变量，即“**export SUMO_HOME=/home/wiotlab/WIoTLab/tools/sumo**”。

(2)利用真实地图生成 NS-2 无线自组网实验场景

①打开浏览器，输入 URL：http://www.openstreetmap.org。在地图上选择自己关注的区域并导出(文件后缀：**.osm**，本章实验文件名：**lab3.osm**)，如图 3.1 所示；

②在 Ubuntu 系统中同时按下快捷键“**Ctrl+Alt+T**”打开终端，然后执行命令“**cd /home/wiotlab/WIoTLab/exps/lab3**”切换到实验目录(**lab3.osm** 已放置在该目录中)，紧接着运行以下命令：

```
$netconvert --osm-files lab3.osm -o lab3.net.xml
$polyconvert --osm-files lab3.osm --net-file lab3.net.xml --type-file $SUMO_HOME/data/typemap/osmPolyconvert.typ.xml -o lab3.poly.xml
```

```
$python $SUMO_HOME/tools/randomTrips.py -n lab3.net.xml -r lab3.rou.xml -e 100 -l
```

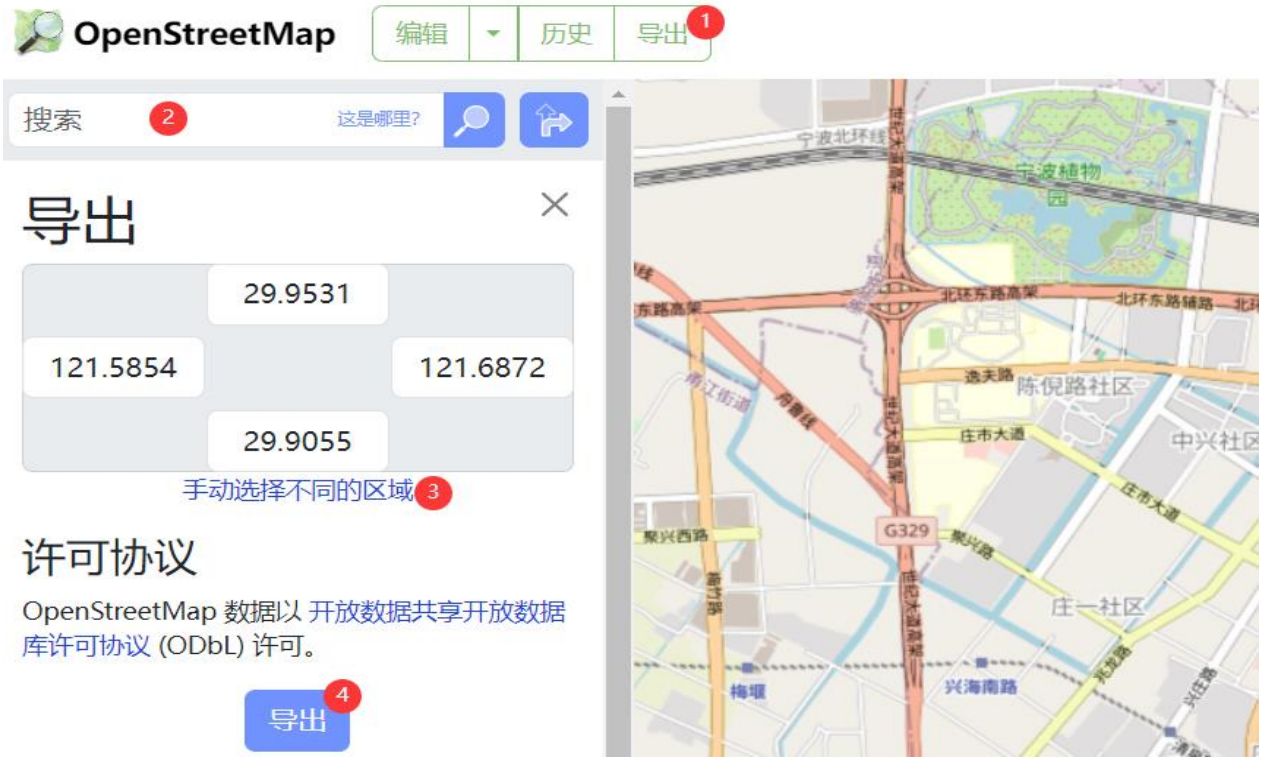


图 3.1　OpenStreetMap 导出地图过程(上图中①→②→③→④)

③在前述实验目录中创建配置文件“**lab3.sumo.cfg**”，并输入下述配置内容，然后执行命令“**sumo-gui lab3.sumo.cfg**”，得到图 3.2 所示的界面。

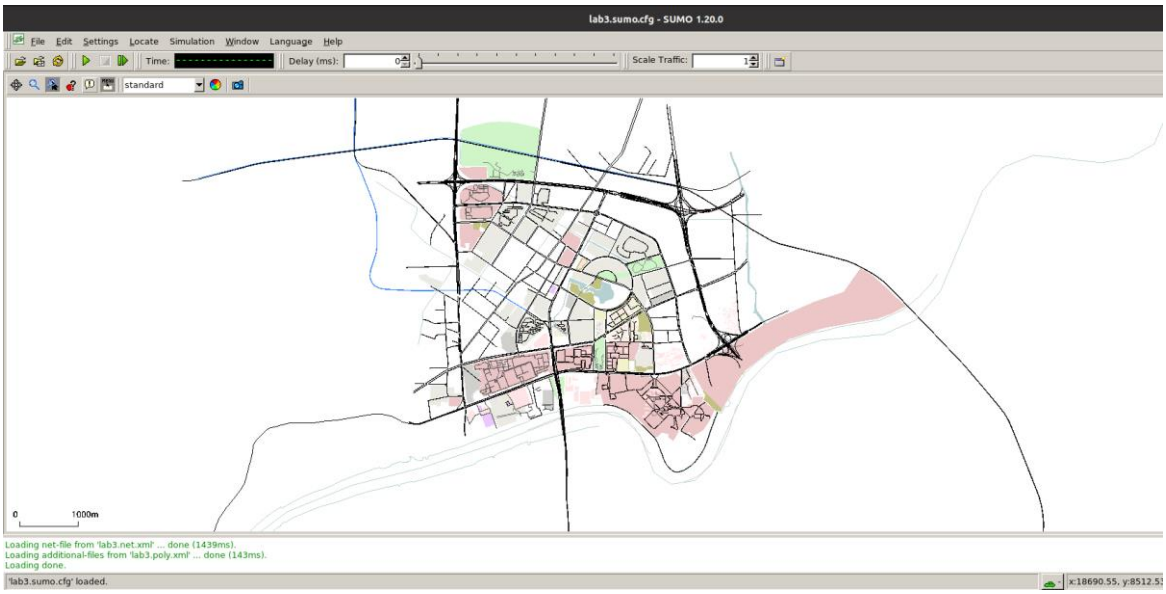


图 3.2　sumo-gui 加载配置后的结果

```
<configuration>
    <input>
```

```
        <net-file value="lab3.net.xml"/>
        <route-files value="lab3.rou.xml"/>
        <additional-files value="lab3.poly.xml"/>
    </input>
    <time>
        <begin value="0"/>
        <end value="100"/>
        <step-length value="0.1"/>
    </time>
</configuration>
```

④导出 NS-2 场景脚本。在前述目录中，终端下执行以下命令，可得到 NS-2 仿真所需的 **lab3_mobility.tcl**，仿真区域大小可从文件 **lab3.tcl** 中获得。

```
$sumo -c lab3.sumo.cfg --fcd-output lab3.sumo.xml
$python $SUMO_HOME/tools/traceExporter.py --fcd-input lab3.sumo.xml --ns2config-output lab3.tcl --ns2activity-output lab3_activity.tcl --ns2mobility-output lab3_mobility.tcl
```

注意，由于 SUMO 生成的 **lab3_mobility.tcl** 中存在负坐标，导致实际使用时会报错，请利用下述的 AWK[1]脚本进行错误纠正。

```
BEGIN {
}
{
    if ((NF > 6) && ($6 <= 0)) next;
    else print $0;
}
END {
}
```

在终端命令行执行下述命令生成最终的场景文件 **lab3_scen.tcl**，假设上述脚本保存的文件名为 **correct_scen.awk**。

```
$awk -f correct_scen.awk lab3_mobility.tcl > lab3_scen.tcl
```

⑤使用生成的网络场景文件，下面以一段简单的仿真代码为例，展示使用方法：

```
set val(chan) Channel/WirelessChannel
set val(prop) Propagation/TwoRayGround
set val(netif) Phy/WirelessPhy
set val(mac) Mac/802_11
set val(ifq) Queue/DropTail/PriQueue
set val(ll) LL
set val(ant) Antenna/OmniAntenna
set val(ifqlen) 50
set val(nn) 92 ;#该值要根据 lab3.tcl 给出的数值来设置
```

[1] https://www.gnu.org/software/gawk/manual/gawk.html

```
set val(rp) AODV
set opt(x) 4707 ;#该值要略大于 lab3.tcl 中给出 x 范围
set opt(y) 3002 ;#该值要略大于 lab3.tcl 中给出 y 范围

set ns_ [new Simulator]
set tracefd [open lab3.tr w]
$ns_ trace-all $tracefd

set namf [open lab3.nam w]
$ns_ namtrace-all-wireless $namf $opt(x) $opt(y)
set topo [new Topography]
$topo load_flatgrid $opt(x) $opt(y)
create-god $val(nn)

$ns_ node-config -adhocRouting $val(rp) \
                 -llType $val(ll) \
                 -macType $val(mac) \
                 -ifqType $val(ifq) \
                 -ifqLen $val(ifqlen) \
                 -antType $val(ant) \
                 -propType $val(prop) \
                 -phyType $val(netif) \
                 -channelType $val(chan) \
                 -topoInstance $topo \
                 -agentTrace ON \
                 -routerTrace ON \
                 -macTrace OFF \
                 -movementTrace ON

for {set i 0} {$i < $val(nn)} {incr i} {
       set node_($i) [$ns_ node]
       $node_($i) random-motion 0
       $ns_ initial_node_pos $node_($i) 20
}
source lab3_scen.tcl ;//务必要创建了节点后再导入

set tcp [new Agent/TCP]
$tcp set class_ 2
set sink [new Agent/TCPSink]
$ns_ attach-agent $node_(0) $tcp
$ns_ attach-agent $node_(22) $sink
$ns_ connect $tcp $sink
set ftp [new Application/FTP]
$ftp attach-agent $tcp
```

```
$ns_ at 10.0 "$ftp start"
for {set i 0} {$i < $val(nn) } {incr i} {
        $ns_ at 100.0 "$node_($i) reset";
}
$ns_ at 100.0 "stop"
$ns_ at 100.01 "$ns_ halt"
proc stop {} {
        global ns_ tracefd
        $ns_ flush-trace
        close $tracefd
}
$ns_ run
```

3.2 实验环境

本章实验需使用 NS-2 官方源码包中自带的 AODV 和 DSR 两个路由协议，同时给出了路由协议的设计思路。除此之外，不需要扩展任何其他的模块。实验环境为 Ubuntu 20.04+VirtualBox 6.18，具体环境配置过程请参照本书导引实验。后续实验无特别说明时，将采用下述地图生成的 NS-2 仿真场景。



图 3.3　SUMO 生成 NS-2 仿真场景使用的真实地图

3.3 实验步骤

3.3.1 路由协议设计

在路由协议设计时，需明确路由类型，如按需路由和地理位置路由在设计时有所不同，地理位置路由没有单独的路由表，利用节点位置信息计算转发的下一跳。需注意，NS-2 中路由

协议设计需完成：路由转发表(地理位置路由忽略)、路由协议头部和路由代理等功能模块，同时修改 NS-2 的 packet.h、priqueue.cc、ns-lib.tcl、ns-packet.tcl、ns-mobilenode.tcl 和 Makefile.in 几个文件。有关表驱动的路由协议设计思路，可参考 Francisco J. Ros 的“Implementing a New Manet Unicast Routing Protocol in NS2”文章[2]，在此不再一一阐述。

由于地理位置易于获取，因而地理位置贪婪转发路由也受到广泛关注，其基本思想：在接收到来自上层的数据包后，源节点会添加一个包含目标 ID 和位置的头部。源节点将数据包发送给距离目标最近的邻居节点。在从邻居节点接收到数据包后，如果存在比自身更接近目标的邻居节点，该节点就会转发数据包。如果没有比自身更接近的邻居节点，即所谓的局部最小值，它将直接丢弃数据包。当数据包到达目的地时，路由过程完成。有关地理位置贪婪转发路由的详细设计过程可参考 Kazuya Sakai 的文章[3]，在此不再详细赘述。

3.3.2 路由协议评价

在无线自组网路由协议性能评价方面，本章将从吞吐量、数据包投递率和归一化路由负载展开，指标具体定义如下：

①**平均吞吐量**：指单位时间内正确接收到的数据包数量，可表示为

$$T_a = R_d*8/(E_t - S_t)$$

其中，T_a 为平均吞吐量，R_d 为接收端成功接收的数据总量，E_t 为传输结束时间，S_t 为传输开始时间。乘以 8 旨在使吞吐量单位为 bit/s，详细的 Awk 分析脚本如下所示。

```
BEGIN {
      recvdSize = 0;
      startTime = 400;
      stopTime = 0;
}
{
     event = $1;
     time = $2;
     pkt_size = $8;
     level = $4;
     if (level == "AGT" && event == "s" && pkt_size >= 512) {
          if (time < startTime) {
               startTime = time;
          }
     }
     if (level == "AGT" && event == "r" && pkt_size >= 512) {
```

[2] https://jiaziyi.com/wp-content/uploads/2015/12/nsrt-howto.pdf
[3] https://kazuyasakai.fpark.tmu.ac.jp/research/ns2_greedy.html

```
            if (time > stopTime) {
                stopTime = time;
            }
            recvdSize += pkt_size;
        }
}
END {
        printf("Average Throughput [kbps] = %.2f\tstartTime=%.2f\tstopTime=%.2f\n",
        (recvdSize/(stopTime-startTime))*(8/1000),startTime,stopTime);
}
```

②**数据包投递率：**指所有目标节点收到的数据包与发送节点发送的数据包数量的比值，投递率越高则路由性能越好。详细的 AWK 分析脚本如下所示。

```
BEGIN {
    sendLine = 0;
    recvLine = 0;
    fowardLine = 0;
    if(mseq==0)
        mseq=10000;
        for(i=0;i<mseq;i++){
            rseq[i]=-1;
            sseq[i]=-1;
        }
}
$0 ~/^s.* AGT/ {
    sendLine++;
}
$0 ~/^r.* AGT/{
    recvLine++;
}
$0 ~/^f.* RTR/ {
    fowardLine++;
}
END {
    printf "Sent: %d, Received: %d, Delivery Ratio:%.4f\n", sendLine, recvLine,
(recvLine/sendLine);
}
```

③**归一化路由负载：**指所有节点路由包的数量与数据包数量的比值，路由负载越高则路由协议性能越差。详细的 AWK 分析脚本如下所示。

```
BEGIN {
    recvd=0;
    rt_pkts=0;
}
```

```
{
    if (( $1 == "r") && ( $7 == "cbr" || $7 =="tcp" ) && ( $4=="AGT" )) recvd++;
    if (($1 == "s" || $1 == "f") && $4 == "RTR" &&
        ($7 =="AODV" || $7 =="message" || $7 =="DSR")) rt_pkts++;
}
END {
    printf("Normalized Routing Load: %.5f\n", rt_pkts/recvd);
}
```

3.3.3 AODV 协议分析

该实验将分析 AODV 路由的性能，评价指标如 3.3.2 节所述，仿真场景来自真实地图数据且节点数为 98(具体可根据 SUMO 生成的结果确定)。具体的实验脚本如下(仿真场景文件由 SUMO 生成，此处不再列出)：

```
set val(chan)      Channel/WirelessChannel       ;# channel type
set val(prop)      Propagation/TwoRayGround      ;# radio-propagation model
set val(netif)    Phy/WirelessPhy                ;# network interface type
set val(mac)       Mac/802_11                      ;# MAC type
set val(ifq)       Queue/DropTail/PriQueue       ;# interface queue type
set val(ll)        LL                              ;# link layer type
set val(ant)       Antenna/OmniAntenna            ;# antenna model
set val(ifqlen) 50                                ;# max packet in ifq
set val(nn)        98                             ;# number of mobilenodes
set val(rp)        AODV                           ;# routing protocol
set val(x)         391                            ;# X dimension of topography
set val(y)         500                            ;# Y dimension of topography
set val(stop)     100.0                           ;# time of simulation end

set ns_ [new Simulator]
set topo            [new Topography]
$topo load_flatgrid $val(x) $val(y)
create-god $val(nn)

set tracefile [open AODV.tr w]
$ns_ trace-all $tracefile

set namfile [open AODV.nam w]
$ns_ namtrace-all $namfile
$ns_ namtrace-all-wireless $namfile $val(x) $val(y)
set chan [new $val(chan)]

$ns_ node-config -adhocRouting    $val(rp) \
                 -llType              $val(ll) \
                 -macType             $val(mac) \
```

```
                        -ifqType             $val(ifq) \
                        -ifqLen              $val(ifqlen) \
                        -antType             $val(ant) \
                        -propType            $val(prop) \
                        -phyType             $val(netif) \
                        -channel             $chan \
                        -topoInstance    $topo \
                        -energyModel "EnergyModel" \
                        -initialEnergy 50 \
                        -txPower 0.09 \
                        -rxPower 0.07 \
                        -idlePower 0.01 \
                        -sleeppower 0.0001 \
                        -transitionPower 0.02 \
                        -transitionTime 0.0005 \
                        -agentTrace        ON \
                        -routerTrace      ON \
                        -macTrace           ON \
                        -movementTrace ON

for {set i 0} {$i < $val(nn) } { incr i } {
      set node_($i) [$ns_ node]
       $ns_ initial_node_pos $node_($i) 20
}
source lab3_scen.tcl

set udp0 [new Agent/UDP]
$ns_ attach-agent $node_(0) $udp0
set null1 [new Agent/Null]
$ns_ attach-agent $node_(97) $null1
$ns_ connect $udp0 $null1
$udp0 set packetSize_ 1500
set cbr0 [new Application/Traffic/CBR]
$cbr0 attach-agent $udp0
$cbr0 set packetSize_ 1000
$cbr0 set rate_ 1.0Mb
$cbr0 set random_ null
$ns_ at 2.0 "$cbr0 start"
$ns_ at $val(stop) "$cbr0 stop"

proc finish {} {
       global ns_ tracefile namfile
       $ns_ flush-trace
       close $tracefile
```

```
        close $namfile
        exec nam AODV.nam &
        exit 0
}
for {set i 0} {$i < $val(nn) } { incr i } {
        $ns_ at $val(stop) "\$node_($i) reset"
}
$ns_ at $val(stop) "$ns_ nam-end-wireless $val(stop)"
$ns_ at $val(stop) "finish"
$ns_ at $val(stop) "puts \"done\" ; $ns_ halt"
$ns_ run
```

根据上述脚本代码，可按如下操作完成实验：

①使用“**Ctrl+Alt+T**”快捷键打开终端，并用“**cd /home/wiotlab/WIoTLab/exps/lab3**”命令切换路径，接着使用“**touch AODV.tcl**”命令创建脚本文件，将上述代码抄录至该文件中；

②按 3.1.5 节的过程，生成实验所需的仿真场景脚本(脚本名与前述保持一致)，修改 **AODV.tcl** 脚本文件，根据实际生成的场景修改其 **val(nn)**的值，同时确保仿真场景文件也放置在 **AODV.tcl** 相同目录；

③在命令行终端运行“**ns AODV.tcl**”执行脚本，成功后会得到 AODV.tr、AODV.nam 等数据文件，其中 AODV.tr 是待处理的记录；

④分析协议平均吞吐量(3.3.2 节对应代码存为 **thp.awk** 文件)、数据包投递率(3.3.2 节对应代码存为 **pdr.awk** 文件)、归一化路由负载(3.3.2 节对应代码存为 **nrl.awk** 文件)，具体如下：

平均吞吐量：在命令行终端运行“**awk -f thp.awk AODV.tr**”，可得到结果“Average Throughput [kbps] = 818.47 startTime=2.00 stopTime=99.99”；

数据包投递率：在命令行终端运行“**awk -f pdr.awk AODV.tr**”，可得到结果“Sent: 12251, Received: 9829, Delivery Ratio:0.8023”；

归一化路由负载：在命令行终端运行“**awk -f nrl.awk AODV.tr**”，可得到结果“Normalized Routing Load: 0.00997”。

3.3.4 DSR 协议分析

该实验将分析 DSR 性能，实验代码与 3.3.3 节相同，但需将 **val(rp)**的值改为 **DSR**、“**set tracefile [open AODV.tr w]**”修改为“**set tracefile [open DSR.tr w]**”、“**set namfile [open AODV.nam w]**”修改为“**set namfile [open DSR.nam w]**”，其他保持不变。具体实验步骤如下：

①使用“**Ctrl+Alt+T**”快捷键打开终端，并用“**cd /home/wiotlab/WIoTLab/exps/lab3**”命令切换路径，接着使用“**touch DSR.tcl**”命令创建脚本文件，将 3.3.3 节实验代码抄录至该文

件中并做相应修改；

②按 3.1.5 节的过程，生成实验所需的仿真场景脚本(脚本名与前述保持一致)，修改 **DSR.tcl** 代码文件，根据实际生成的场景修改其 **val(nn)**的值，同时确保仿真场景文件也放置在 **DSR.tcl** 相同目录；

③在命令行终端运行“**ns DSR.tcl**”执行脚本，成功后会得到 DSR.tr、DSR.nam 等数据文件，其中 DSR.tr 是待处理的记录；

④分析协议平均吞吐量(3.3.2 节对应代码存为 **thp.awk** 文件)、数据包投递率(3.3.2 节对应代码存为 **pdr.awk** 文件)、归一化路由负载(3.3.2 节对应代码存为 **nrl.awk** 文件)，具体如下：

平均吞吐量：在命令行终端运行“**awk -f thp.awk DSR.tr**”，可得到结果“Average Throughput [kbps] = 802.60 startTime=2.00 stopTime=99.99”；

数据包投递率：在命令行终端运行“**awk -f pdr.awk DSR.tr**”，可得到结果“Sent: 12251, Received: 9831, Delivery Ratio:0.8025”；

归一化路由负载：在命令行终端运行“**awk -f nrl.awk DSR.tr**”，可得到结果“Normalized Routing Load: 0.00020”。

与 AODV 相比，部署 DSR 协议时平均吞吐略低，但是数据包投递率有少量提升，且协议本身带来的负载更低。

3.4 扩展练习*

在本章中，主要给出了无线自组网路由的基本概念和设计思路，并展示了如何用真实地图数据生成 NS-2 仿真场景。基于生成场景数据，设计实验脚本完整分析了 AODV 和 DSR 两个协议的性能。读者可在本章实验基础上从以下几个方面进行扩展：

(1)对比分析更多的无线自组网路由协议，能更加全面了解不同应用场景的协议性能；

(2)利用本章给出的 SUMO 生成 NS-2 仿真场景的步骤，生成更多的场景进行分析；

(3)发散思维，扩展更多的路由性能评价指标，如能耗、比特代价等

参考文献

[1] K. Akkaya, M. Younis. A Survey on Routing Protocols for Wireless Sensor Networks. Ad Hoc Networks, 2005, 3(3): 325-349.

[2] S. Mohapatra, P. Kanungo. Performance Analysis of AODV, DSR, OLSR and DSDV Routing Protocols using NS2 Simulator. Procedia Engineering, 2012, 30: 69-76.